Flashes can store more than just text messages—you can use them to pass all kinds of information between actions. Obviously, for longer-term information you'd want to use the session (probably in conjunction with your database) to store the data, but the flash is great if you want to pass parameters from one request to the next.

Because the flash data is stored in the session, all the usual rules apply. In particular, every object must be serializable. We strongly recommend passing only basic objects like Strings or Hashes in the flash.

Callbacks

Callbacks enable you to write code in your controllers that wrap the processing performed by actions—you can write a chunk of code once and have it be called before or after any number of actions in your controller (or your controller's subclasses). This turns out to be a powerful facility. Using callbacks, we can implement authentication schemes, logging, response compression, and even response customization.

Rails supports three types of callbacks: before, after, and around. Such callbacks are called just prior to and/or just after the execution of actions. Depending on how you define them, they either run as methods inside the controller or are passed to the controller object when they are run. Either way, they get access to details of the request and response objects, along with the other controller attributes.

Before and After Callbacks

As their names suggest, before and after callbacks are invoked before or after an action. Rails maintains two chains of callbacks for each controller. When a controller is about to run an action, it executes all the callbacks on the before chain. It executes the action before running the callbacks on the after chain.